ALGORITHM	SCENARIO	SHORT DEFINITION
**Euclid’s algorithm	Designing a 1920x1080 grid layout in figma into the largest possible perfectly square button without any leftover space	Finds the greatest common divisor (GCD) of two numbers by repeatedly replacing the larger number with the remainder of its division by the smaller number until the remainder is zero.
**Consecutive integer check algorithm	Designing a 1920x1080 grid layout in figma into the largest possible perfectly square buttons without any leftover space	Finds the GCD by checking all integers from $min(m, n)$ downward until a common divisor is found.
**Middle school procedure	Designing a 1920x1080 grid layout in figma into the largest possible perfectly square buttons without any leftover space	Finds the GCD by identifying common prime factors of two numbers and multiplying them together.
SequentialSearch	Finding the correct jeep to take in a jeepney terminal	Scans a list linearly from start to finish to find a target value or determine its absence.
MaxElement	Finding who got the highest score in the alco quiz who will treat the whole class	Iterates through an array while maintaining a variable that tracks the largest value encountered.
UniqueElements	Used in unique username login systems for the database system in VS code	Checks if all elements in a collection are distinct by comparing each pair or using a frequency tracker.
Factorial	Woke up late; have to do a certain number of chores before leaving the chores. Which one to do first?	Calculates the product of all positive integers up to n ($n!$), often implemented via recursion or iteration.
Selection Sort	Drag and dropping unorganized and cluttered UI icons for your figma project by file size one by one.	Repeatedly finds the minimum element from the unsorted portion of a list and swaps it into its correct position.
**Bubble Sort	Compiling a massive list of user data for a management system. Before you can test the search functions, you need to sort the thousands of entries by ID.	Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
BF String Matching	Finding your friend to add him in the rpg game	(Brute Force) Checks for a pattern within a text by aligning it at every possible position until a match is found.
**DFS	Unlocking a specific node in a character skill tree in an rpg game.	(Depth-First Search) Explores a graph by visiting a node and then recursively visiting all nodes along a single branch as deep as possible before backtracking.
**BFS	Unlocking a specific node in a character skill tree in an rpg game.	(Breadth-First Search) Explores a graph layer by layer, visiting all neighbors of a node before moving to the next level of depth.
Insertion Search	Organizing your flashcards for your upcoming ethics enumeration quiz.	(Often synonymous with Insertion Sort's search) Locates the correct position for an element within a sorted sub-list to maintain order.
**Binary search	Finding your favorite OPM song in a karaoke songbook	Efficiently finds a target in a sorted array by repeatedly dividing the search interval in half.
**Interpolation search	Finding your favorite OPM song in a karaoke songbook	An improvement on binary search for uniformly distributed sorted data that estimates the target's position based on its value.
**Merge sort	Compiling a massive list of user data for a management system. Before you can test the search functions, you need to sort the thousands of entries by ID.	A divide-and-conquer algorithm that splits a list into halves, sorts them recursively, and merges the sorted halves back together.
**Quicksort	Compiling a massive list of user data for a management system. Before you can test the search functions, you need to sort the thousands of entries by ID.	A divide-and-conquer algorithm that picks a "pivot" and partitions the array into elements smaller and larger than the pivot.
***Heapsort	Compiling a massive list of user data for a management system. Before you can test the search functions, you need to sort the thousands of entries by ID.	A comparison-based sorting technique that builds a Binary Heap and repeatedly extracts the maximum/minimum element.
***Coin Row	You’re standing in front of a vintage vending machine in the basement of the engineering wing. You’ve just finished a project and have a handful of different coins. You need to provide the exact amount for a bag of chips using the smallest number of coins possible. While sitting on a long bench nearby, you notice someone dropped a line of coins; however, a sensor prevents you from picking up any two coins that are directly adjacent. Later, you play a quick grid-based mobile game where	A dynamic programming problem to find the maximum value of coins picked from a row, provided no two adjacent coins are chosen.
***Change Making		Finds the minimum number of coins (of specific denominations) needed to make a target total.
***Coin Collecting		A dynamic programming task to find the path in a grid that collects the maximum number of coins while moving only right or down.

	you must move a character from one corner of a room to the other, grabbing as much loot as possible.	
***Prim's	You wake up late and need to get to the Computer Science building. You're looking at a map of the campus walkways, noting that some paths are crowded with students (high weight) while others are clear (low weight). Once you finally arrive at the student lounge, your study group asks you to help them connect several workstations using the minimum amount of wiring possible to save on costs, ensuring every desk is linked to the network without any unnecessary loops.	A greedy algorithm that builds a Minimum Spanning Tree (MST) by starting from a single node and adding the cheapest edge to a new vertex.
***Kruskal		A greedy algorithm that builds an MST by sorting all edges by weight and adding the cheapest ones that do not form a cycle.
***Djikstra		Finds the shortest path from a starting node to all other nodes in a weighted graph with non-negative edge weights.
***Huffman	It's 11:00 PM and you are finalizing a massive documentation file that needs to be compressed to fit the submission portal's 5MB limit. Meanwhile, you are also debugging a complex logic circuit; you find yourself trying a specific configuration, hitting a dead end where the logic fails, and having to step back to the last working state to try a different lead. Finally, you're trying to pack your essentials into a small tech pouch for tomorrow's presentation, quickly discarding certain combinations of chargers and cables the moment you realize they will be too heavy or won't fit.	A greedy algorithm used for lossless data compression that assigns shorter binary codes to more frequent characters.
***Backtracking		A systematic trial-and-error method that builds a solution incrementally and "backtracks" as soon as a partial solution violates constraints.
***Branch and Bound		An optimization technique that explores a state-space tree but uses "bounds" to prune branches that cannot yield a better solution than the current best.

Mathematical & Foundation

- **Euclid's Algorithm:** Number Theory
- **Consecutive Integer Check:** Brute Force Math
- **Middle School Procedure:** Factorization
- **Factorial:** Recursion/Counting

Searching & Pattern Matching

- **Sequential Search:** Linear Search
- **Binary Search:** Divide & Conquer
- **Interpolation Search:** Improved Search
- **MaxElement:** Iteration
- **UniqueElements:** Validation
- **BF String Matching:** String Processing

Sorting

- **Selection Sort:** Simple Sort
- **Bubble Sort:** Simple Sort
- **Merge Sort:** Stable Sort
- **Quicksort:** Efficient Sort
- **Heapsort:** Priority Sort

Graph & Network

- **DFS (Depth-First Search):** Graph Traversal
- **BFS (Breadth-First Search):** Graph Traversal
- **Prim's:** Greedy (Minimum Spanning Tree)
- **Kruskal's:** Greedy (Minimum Spanning Tree)
- **Dijkstra's:** Greedy (Shortest Path)

Dynamic Programming & Optimization

- **Coin Row:** Dynamic Programming
- **Change Making:** Optimization
- **Coin Collecting:** Dynamic Programming (Grid)
- **Huffman Coding:** Greedy/Compression

State-Space Search

- **Backtracking:** Exhaustive Search
- **Branch and Bound:** Pruned Search